PROJECT

Faster R-CNN OBB Project - Phase 1 Notes

Dataset Understanding and Exploration

Project Objective

The objective of this project is to build an Object Detection and Classification model using Faster R-CNN with Oriented Bounding Boxes (OBB) on the DOTA dataset.

The model should be able to:

1. **Detect objects in aerial images.**
2. **Classify detected objects into their respective classes.**
3. **Predict rotated bounding boxes (OBB) instead of normal horizontal bounding boxes.**

Dataset Used

Dataset: DOTA v1.0 Validation Set

Number of Images:

458

Number of Label Files:

458

Image Format:

PNG

Annotation Format:

TXT

Understanding DOTA Annotations

Each label file contains:

Example:

imagesource:GoogleEarth
gsd:0.115726939386

937 913 921 912 923 874 940 875 small-vehicle 0

Metadata

imagesource

Example:

imagesource:GoogleEarth

Indicates the source of the image.

Not used during training.

---

gsd

Example:

gsd:0.115726939386

Ground Sample Distance.

Represents real-world distance covered by one pixel.

Not used in the initial training phase.

Object Annotation Format

Example:

937 913 921 912 923 874 940 875 small-vehicle 0

This annotation contains:

Rotated Bounding Box Coordinates

Point 1:

(937, 913)

Point 2:

(921, 912)

Point 3:

(923, 874)

Point 4:

(940, 875)

These four points define an Oriented Bounding Box (OBB).

Class Name

small-vehicle

Represents the object category.

Difficulty Flag

0

Meaning:

0 = Normal Object

1 = Difficult Object

Dataset Structure

Project Structure:

FasterRCNN_OBB_Project/

├── data/

│ ├── images/

│ │ ├── P0003.png

│ │ ├── P0004.png

│ │ └── ...

│ │

│ └── labels/

│ ├── P0003.txt

│ ├── P0004.txt

│ └── ...

│

├── check_dataset.py

└── dataset_prepare.py

Step 1 - Verify Dataset Access

Goal:

Ensure Python can access images and labels.

Code:

```
import os

image_dir = "data/images"
label_dir = "data/labels"

images = os.listdir(image_dir)
labels = os.listdir(label_dir)

print("Number of images:", len(images))
print("Number of labels:", len(labels))

print("First image:", images[0])
print("First label:", labels[0])
```

Purpose:

- **Verify image folder exists.**
- **Verify label folder exists.**
- **Count images and labels.**

Output:

Number of images: 458

Number of labels: 458

Step 2 - Read Annotation File

Goal:

Open and inspect one DOTA label file.

Code:

```
with open("data/labels/P0003.txt", "r") as file:
lines = file.readlines()

print("Total lines:", len(lines))

for line in lines[:5]:
print(line.strip())
```

Purpose:

- **Open annotation file.**
- **Read all lines.**
- **Inspect annotation format.**

Output:

Total lines: 57

This indicates:

2 metadata lines

55 object annotations

Step 3 - Parse One Object

Goal:

Extract information from one annotation.

Code:

```
with open("data/labels/P0003.txt", "r") as file:
lines = file.readlines()

object_line = lines[2]

parts = object_line.split()

points = list(map(int, parts[:8]))

class_name = parts[8]

difficulty = int(parts[9])

print(points)
print(class_name)
print(difficulty)
```

Purpose:

Extract:

- **Bounding Box Points**
- **Class Name**
- **Difficulty Flag**

Output:

Points:

```
[937, 913, 921, 912, 923, 874, 940, 875]
```

Class:

small-vehicle

Difficulty:

0

Step 4 - Parse Multiple Objects

Goal:

Read all object annotations from a label file.

Code:

```
with open("data/labels/P0003.txt", "r") as file:
lines = file.readlines()

object_lines = lines[2:]

print("Number of objects:", len(object_lines))

for line in object_lines[:5]:

parts = line.split()


points = list(map(int, parts[:8]))

class_name = parts[8]

difficulty = int(parts[9])


print("Points:", points)

print("Class:", class_name)
```

print("Difficulty:", difficulty)

Purpose:

- **Skip metadata.**
- **Read all objects.**
- **Parse annotations.**

Output:

Number of objects: 55

Step 5 - Dataset Class Distribution

Goal:

Find all classes and count objects.

Code:

```
import os

label_folder = "data/labels"

class_counts = {}

all_labels = os.listdir(label_folder)

for label_file in all_labels:

file_path = os.path.join(label_folder, label_file)


with open(file_path, "r") as file:

    lines = file.readlines()
```

```
object_lines = lines[2:]

for line in object_lines:

    parts = line.split()

    class_name = parts[8]

    if class_name not in class_counts:
        class_counts[class_name] = 0

    class_counts[class_name] += 1
print(class_counts)
```

Purpose:

Count objects per class.

Dataset Statistics

Class Counts:

ship : 8960

small-vehicle : 5438

large-vehicle : 4387

storage-tank : 2888

plane : 2531

harbor : 2090

tennis-court : 760

bridge : 464

swimming-pool : 440

baseball-diamond : 214

roundabout : 179

soccer-ball-field : 153

ground-track-field : 144

basketball-court : 132

helicopter : 73

Approximate Total Objects:

28,853

Step 6 - Create Class Mapping

Goal:

Convert class names into numeric labels.

Code:

```
import os
```

```
label_folder = "data/labels"

classes = set()

for label_file in os.listdir(label_folder):

file_path = os.path.join(label_folder, label_file)


with open(file_path, "r") as file:

    lines = file.readlines()


for line in lines[2:]:


    parts = line.split()


    class_name = parts[8]


    classes.add(class_name)

for i, class_name in enumerate(sorted(classes)):
print(f"{class_name} -> {i}")
```

Output:

```
baseball-diamond -> 0

basketball-court -> 1

bridge -> 2

ground-track-field -> 3
```

harbor -> 4

helicopter -> 5

large-vehicle -> 6

plane -> 7

roundabout -> 8

ship -> 9

small-vehicle -> 10

soccer-ball-field -> 11

storage-tank -> 12

swimming-pool -> 13

tennis-court -> 14

Total Classes:

15

Key Learnings

1. **DOTA uses Oriented Bounding Boxes (OBB).**
2. **Each object is represented by 4 corner points.**
3. **Each annotation contains:**
    - **8 coordinates**
    - **1 class label**
    - **1 difficulty flag**

4. **Dataset contains:**
    - **458 images**
    - **458 labels**
    - **15 classes**
    - **~28,853 objects**
5. **Before training a detector, dataset exploration and verification are essential.**
6. **Class names must be converted into numerical IDs before training.**

Current Project Status

Dataset Understanding: Completed

Dataset Exploration: Completed

Class Mapping: Completed

Next Phase:

Dataset Preparation and Dataset Loader Construction

Faster R-CNN OBB Project - Phase 2 Notes

Dataset Preparation

Objective

The goal of this phase is to convert raw DOTA dataset files into a structured format that can be used by a Faster R-CNN object detection model.

At the end of this phase we should have:

- **Verified dataset integrity**
- **Created class mappings**
- **Built a dataset loader**
- **Converted annotations into machine-readable format**

Why Dataset Preparation Is Important

Raw files cannot be directly used by a neural network.

Current Dataset:

P0003.png

P0003.txt

P0004.png

P0004.txt

...

Neural networks require structured numerical data.

Target Format:

{

"image_path": "...",

"boxes": [...],

"labels": [...]

}

Dataset preparation converts raw annotations into this format.

Step 1 - Verify Image-Label Pairing

Goal:

Ensure every image has a corresponding label file.

Problem:

If:

P0003.png

exists but:

P0003.txt

does not exist,

training will fail.

Therefore dataset consistency must be verified.

Code

import os

```
image_folder = "data/images"
label_folder = "data/labels"

images = set()

for image_file in os.listdir(image_folder):
    image_name = os.path.splitext(image_file)[0]
    images.add(image_name)

labels = set()

for label_file in os.listdir(label_folder):
    label_name = os.path.splitext(label_file)[0]
    labels.add(label_name)

print("Images:", len(images))
print("Labels:", len(labels))

missing_labels = images - labels
missing_images = labels - images
```

print("\nMissing Labels:", len(missing_labels))

print("Missing Images:", len(missing_images))

Output

Images: 458

Labels: 458

Missing Labels: 0

Missing Images: 0

Conclusion

Dataset is synchronized.

Every image has a label file.

Every label file has an image.

Dataset is safe for training.

Step 2 - Create Class Mapping

Goal:

Convert textual class names into numerical IDs.

Neural networks cannot process strings such as:

"ship"

"plane"

"harbor"

Instead they require:

9

7

4

---

config.py

```
CLASS_TO_ID = {
    "baseball-diamond": 0,
    "basketball-court": 1,
    "bridge": 2,
    "ground-track-field": 3,
    "harbor": 4,
    "helicopter": 5,
    "large-vehicle": 6,
    "plane": 7,
    "roundabout": 8,
    "ship": 9,
    "small-vehicle": 10,
    "soccer-ball-field": 11,
```

```
    "storage-tank": 12,
    "swimming-pool": 13,
    "tennis-court": 14
}

ID_TO_CLASS = {v: k for k, v in
CLASS_TO_ID.items()}

NUM_CLASSES = len(CLASS_TO_ID)
```

Understanding CLASS_TO_ID

Example:

```
CLASS_TO_ID["ship"]
```

Output:

```
9
```

Meaning:

ship is represented internally by class ID 9.

Understanding ID_TO_CLASS

Example:

```
ID_TO_CLASS[9]
```

Output:

"ship"

Used during prediction and visualization.

Understanding NUM_CLASSES

NUM_CLASSES = 15

Meaning:

The classifier head of Faster R-CNN must predict 15 classes.

Step 3 - Build Dataset Loader

Goal:

Read all images and annotations.

Convert them into Python objects.

Dataset Loader Code

```
import os

from config import CLASS_TO_ID

image_folder = "data/images"
label_folder = "data/labels"
```

```
dataset = []

all_images = sorted(os.listdir(image_folder))

for image_file in all_images:

    image_name = os.path.splitext(image_file)[0]

    label_file = image_name + ".txt"

    label_path = os.path.join(label_folder, label_file)

    boxes = []
    labels = []

    with open(label_path, "r") as file:
        lines = file.readlines()

    for line in lines[2:]:

        parts = line.split()
```

```
            points = list(map(int, parts[:8]))

            class_name = parts[8]

            class_id = CLASS_TO_ID[class_name]

            boxes.append(points)

            labels.append(class_id)

        sample = {
            "image_path": os.path.join(image_folder, image_file),
            "boxes": boxes,
            "labels": labels
        }

        dataset.append(sample)

print("Total Samples:", len(dataset))
```

print(dataset[0])

Understanding the Dataset Loader

The loader performs the following operations:

1. **Reads all images.**
2. **Finds corresponding label files.**
3. **Reads DOTA annotations.**
4. **Extracts OBB coordinates.**
5. **Converts class names to numerical IDs.**
6. **Stores everything in a structured format.**

Structure of One Dataset Sample

Example:

```
{
   "image_path": "data/images/P0003.png",

   "boxes": [
      [937,913,921,912,923,874,940,875],
      [638,959,638,935,694,939,693,962]
   ],
```

```
    "labels": [
        10,
        6
    ]
}
```

Meaning of boxes

Example:

[937,913,921,912,923,874,940,875]

Represents:

(x1,y1)

(x2,y2)

(x3,y3)

(x4,y4)

The four corners of an Oriented Bounding Box.

Meaning of labels

Example:

10

Using CLASS_TO_ID:

10 → small-vehicle

Example:

6

Using CLASS_TO_ID:

6 → large-vehicle

Output

Total Samples: 458

Meaning:

All 458 images were successfully loaded.

Key Learnings

1. **Dataset integrity must be verified before training.**
2. **Neural networks cannot process text labels.**
3. **Class names must be converted into numerical IDs.**
4. **A dataset loader converts raw files into structured Python objects.**
5. **Each image is represented as:**

{

"image_path": ...,

"boxes": ...,

"labels": ...

}

6. OBB annotations are currently stored as 8 corner coordinates.
7. Dataset loader successfully loads all 458 images and all object annotations.

Current Project Status

Dataset Understanding: Completed

Dataset Exploration: Completed

Image-Label Verification: Completed

Class Mapping: Completed

Dataset Loader: Completed

Next Phase

Phase 3: Dataset Visualization

Objectives:

1. Load image files.
2. Draw Oriented Bounding Boxes (OBB).
3. Verify annotation correctness visually.
4. Understand how OBBs appear on aerial images.

5. **Prepare for OBB mathematical conversion.**

Phase 3 will be the first visual stage of the project.

Faster R-CNN OBB Project - Phase 3 Notes

Dataset Visualization and Annotation Verification

Objective

The objective of this phase is to visually verify the correctness of DOTA annotations before training a Faster R-CNN model.

Visualization is an important step because annotation errors can significantly affect model performance.

This phase helps ensure that:

- **Images are loaded correctly.**
- **Bounding box coordinates are parsed correctly.**
- **OBB annotations match actual objects.**
- **Class labels are correctly assigned.**

Why Visualization Is Important

Before training any object detection model, researchers verify the annotations visually.

Reason:

Incorrect annotations can lead to:

- **Poor training**
- **Incorrect predictions**
- **Low accuracy**
- **Difficult debugging**

Visualization allows us to confirm that the dataset parser is functioning correctly.

Understanding OBB Visualization

DOTA annotations are stored as:

x1 y1 x2 y2 x3 y3 x4 y4 class difficulty

Example:

937 913 921 912 923 874 940 875 small-vehicle 0

These coordinates represent the four corners of an Oriented Bounding Box (OBB).

Unlike normal bounding boxes, OBBs can rotate according to the orientation of the object.

Step 1 - Install OpenCV

Goal:

Install OpenCV for image processing and visualization.

Command

pip install opencv-python

Output

Successfully installed opencv-python

Purpose

OpenCV provides functions for:

- **Reading images**
- **Drawing polygons**
- **Writing text**
- **Saving images**

Step 2 - Draw One OBB

Goal:

Draw a single Oriented Bounding Box on an image.

Code

import cv2

import numpy as np

```
image = cv2.imread("data/images/P0003.png")

points = np.array([
    [937, 913],
    [921, 912],
    [923, 874],
    [940, 875]
], dtype=np.int32)

cv2.polylines(
    image,
    [points],
    isClosed=True,
    color=(0, 255, 0),
    thickness=2
)

cv2.imwrite("output.png", image)

print("Image saved as output.png")
```

Purpose

This code:

1. **Loads an image.**
2. **Creates a polygon from four corner points.**
3. **Draws the polygon.**
4. **Saves the result.**

Output

output.png

Result:

A single green OBB is drawn on the corresponding object.

Understanding cv2.polylines()

Function:

cv2.polylines()

Purpose:

Draws a polygon by connecting points.

Example:

Point1 → Point2

Point2 → Point3

Point3 → Point4

Point4 → Point1

Result:

A complete OBB is formed.

Step 3 - Draw All OBBs

Goal:

Visualize all object annotations for a single image.

Code

```
import cv2
import numpy as np

image = cv2.imread("data/images/P0003.png")

with open("data/labels/P0003.txt", "r") as file:
    lines = file.readlines()

for line in lines[2:]:

    parts = line.split()
```

```
    points = np.array([
        [int(parts[0]), int(parts[1])],
        [int(parts[2]), int(parts[3])],
        [int(parts[4]), int(parts[5])],
        [int(parts[6]), int(parts[7])]
    ], dtype=np.int32)

    cv2.polylines(
        image,
        [points],
        isClosed=True,
        color=(0, 255, 0),
        thickness=1
    )

cv2.imwrite("all_boxes.png", image)

print("Saved as all_boxes.png")
```

Purpose

This code:

1. **Reads all annotations.**
2. **Extracts OBB coordinates.**
3. **Draws every OBB.**
4. **Saves the visualization.**

Output

all_boxes.png

Result:

55 OBBs were successfully drawn for image P0003.

Verification Results

Observed:

- **School buses correctly enclosed.**
- **Road vehicles correctly enclosed.**
- **Parked vehicles correctly enclosed.**
- **Rotation angles accurately represented.**

Conclusion:

Annotation parser is functioning correctly.

Step 4 - Draw Class Labels

Goal:

Display object class names along with bounding boxes.

Code

```
import cv2
import numpy as np

image = cv2.imread("data/images/P0003.png")

with open("data/labels/P0003.txt", "r") as file:
    lines = file.readlines()

for line in lines[2:]:

    parts = line.split()

    points = np.array([
        [int(parts[0]), int(parts[1])],
        [int(parts[2]), int(parts[3])],
        [int(parts[4]), int(parts[5])],
        [int(parts[6]), int(parts[7])]
```

```
], dtype=np.int32)

class_name = parts[8]

cv2.polylines(
    image,
    [points],
    isClosed=True,
    color=(0, 255, 0),
    thickness=1
)

cv2.putText(
    image,
    class_name,
    (int(parts[0]), int(parts[1])),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.4,
    (0, 0, 255),
    1
)
```

cv2.imwrite("labeled_boxes.png", image)

print("Saved as labeled_boxes.png")

Understanding cv2.putText()

Function:

cv2.putText()

Purpose:

Writes text on an image.

Example:

large-vehicle

or

small-vehicle

appears next to the bounding box.

Output

labeled_boxes.png

Result:

Each OBB is displayed together with its corresponding class name.

Verification Results

Observed:

- **School buses labeled as large-vehicle.**
- **Cars labeled as small-vehicle.**
- **Bounding boxes matched objects correctly.**
- **Class labels matched objects correctly.**

Conclusion:

Class extraction and annotation parsing are functioning correctly.

Key Learnings

1. **Visualization is essential before training.**
2. **OpenCV can be used to:**
    - **Read images**
    - **Draw polygons**
    - **Draw text**
    - **Save images**
3. **DOTA annotations represent OBBs using four corner points.**
4. **OBBs provide tighter localization than horizontal bounding boxes.**
5. **Visualization confirms:**

- Coordinate correctness
- Parser correctness
- Class label correctness

6. Incorrect annotations can be detected visually before training.

Difference Between HBB and OBB

Horizontal Bounding Box (HBB):

```
+-------------+
|   Object    |
+-------------+
```

Characteristics:

- Axis-aligned
- Cannot rotate
- Includes extra background

Oriented Bounding Box (OBB):

```
   /--------/
  / Object /
 /--------/
```

Characteristics:

- Rotated

- **Fits object tightly**
- **Better for aerial imagery**

Current Project Status

Phase 1: Dataset Understanding Completed

Phase 2: Dataset Preparation Completed

Phase 3: Dataset Visualization Completed

Next Phase

Phase 4: OBB Mathematics

Objectives:

1. **Understand polygon representation.**
2. **Understand rotated rectangle representation.**
3. **Compute:**
    - **Center (cx, cy)**
    - **Width (w)**
    - **Height (h)**
    - **Rotation Angle (θ)**
4. **Convert DOTA annotations:**

x1,y1,x2,y2,x3,y3,x4,y4

into:

(cx, cy, w, h, θ)

5. **Prepare annotations for Rotated Faster R-CNN training.**

Phase 4 introduces the mathematical foundation of oriented object detection.

Faster R-CNN OBB Project - Phase 4 Notes

OBB Mathematics and Annotation Conversion

Objective

The objective of this phase is to understand how Oriented Bounding Boxes (OBBs) are represented mathematically and how DOTA polygon annotations can be converted into a format suitable for Rotated Faster R-CNN training.

This phase forms the mathematical foundation of the entire project.

Why Is OBB Mathematics Important?

DOTA annotations are stored as:

x1 y1 x2 y2 x3 y3 x4 y4

Example:

638 959 638 935 694 939 693 962

This representation uses four corner points.

However, most rotated object detectors do not directly predict these eight coordinates.

Instead, they predict:

(cx, cy, w, h, θ)

where:

cx = center x coordinate

cy = center y coordinate

w = width

h = height

θ = rotation angle

This representation is easier for neural networks to learn.

Polygon Representation

DOTA stores an object using four corner points.

Example:

638 959

638 935

694 939

693 962

This is called:

Polygon OBB Representation

Stored as:

[x1,y1,x2,y2,x3,y3,x4,y4]

Example:

[638,959,638,935,694,939,693,962]

Rotated Rectangle Representation

Instead of storing four corners, we store:

(cx,cy,w,h,theta)

Example:

(665.75,948.75,56.14,24,4.09)

This representation is used during model training.

Step 1 - Center Calculation

The center of the OBB is computed using the average of all four corners.

Formula:

$$c_x=\frac{x_1+x_2+x_3+x_4}{4}$$

$$c_y=\frac{y_1+y_2+y_3+y_4}{4}$$

Example

Coordinates:

638

638

694

693

Center X:

(638+638+694+693)/4

= 665.75

Coordinates:

959

935

939

962

Center Y:

(959+935+939+962)/4

= 948.75

Result

Center

(665.75, 948.75)

Step 2 - Width Calculation

The length of one edge is calculated using the distance formula.

Formula:

Example

Points:

(638,935)

(694,939)

Calculation:

Δx = 56

Δy = 4

Distance

$= \sqrt{(56^2 + 4^2)}$

$= \sqrt{3152}$

$= 56.14$

Step 3 - Height Calculation

Using another edge:

Points:

(638,959)

(638,935)

Calculation:

$\Delta x = 0$

$\Delta y = 24$

Distance

$= \sqrt{(0^2 + 24^2)}$

$= 24$

Width and Height Selection

Rule:

Longer Side → Width

Shorter Side → Height

Result:

Width = 56.14

Height = 24

Step 4 - Angle Calculation

Angle determines object orientation.

Formula:

Example

$\Delta y = 4$

$\Delta x = 56$

$\theta = \tan^{-1}(4/56)$

≈ 4.09°

Result

Angle = 4.09°

Complete OBB Representation

For:

638 959

638 935

694 939

693 962

Final rotated representation becomes:

(cx,cy,w,h,θ)

(665.75,948.75,56.14,24,4.09)

Why Angle Sometimes Becomes Negative

Example:

-90°

or

-177°

This happens because:

atan2()

returns angles based on direction.

Examples:

179°

≈

-181°

Both represent almost the same orientation.

This is known as:

Angle Ambiguity

and is a common challenge in rotated object detection.

Step 5 - Automatic OBB Conversion

Manual calculations are not practical for:

458 images

28,853 objects

Therefore a conversion function was created.

Code

```
import math

def polygon_to_obb(points):

    x1,y1,x2,y2,x3,y3,x4,y4 = points

    cx = (x1+x2+x3+x4)/4
    cy = (y1+y2+y3+y4)/4

    side1 = math.sqrt(
        (x2-x1)**2 +
        (y2-y1)**2
    )

    side2 = math.sqrt(
        (x3-x2)**2 +
        (y3-y2)**2
    )

    width = max(side1, side2)
```

```
height = min(side1, side2)

if side1 > side2:

    theta = math.degrees(
        math.atan2(
            y2-y1,
            x2-x1
        )
    )

else:

    theta = math.degrees(
        math.atan2(
            y3-y2,
            x3-x2
        )
    )

return cx,cy,width,height,theta
```

Purpose

Input:

[638,959,638,935,694,939,693,962]

Output:

(665.75,948.75,56.14,24,4.09)

Step 6 - Convert Real Objects

The conversion function was tested on actual DOTA objects.

Example Output:

Object 1

Class: small-vehicle

Rotated Box:

(930.25,893.5,38.05,16.03,-86.99)

Object 2

Class: large-vehicle

Rotated Box:

(665.75,948.75,56.14,24,4.09)

Verification:

Manual calculations matched automatic calculations.

Conclusion:

Conversion Logic Verified

Step 7 - Build Dataset Structure

Goal:

Store both representations together.

Why Store Both?

Polygon Box:

[x1,y1,x2,y2,x3,y3,x4,y4]

Used for:

Visualization

Drawing OBBs

DOTA Evaluation

Rotated Box:

(cx,cy,w,h,theta)

Used for:

Training

Regression Targets

Rotated Faster R-CNN

Final Dataset Structure

```
{
    "image_path": "...",

    "polygon_boxes": [...],

    "rotated_boxes": [...],

    "labels": [...]
```

}

Dataset Construction

The dataset builder performed:

Image

↓

Read Label File

↓

Extract Polygon

↓

Convert To OBB

↓

Store Class ID

↓

Create Sample

for all images.

Result

Output:

Total Images: 458

Dataset successfully built.

Key Learnings

1. **DOTA stores OBBs as polygons.**
2. **Neural networks usually learn:**

(cx,cy,w,h,θ)

instead of 8 corner coordinates.

3. **Center is calculated using the average of four points.**
4. **Width and height are calculated using Euclidean distance.**
5. **Angle is computed using:**

atan2()

6. **Polygon annotations can be automatically converted to rotated boxes.**
7. **Both polygon and rotated representations should be stored.**
8. **A complete dataset structure was created for all 458 images.**

Phase 4 Status

Polygon Representation ✅

Center Calculation ✅

Width Calculation ✅

Height Calculation ✅

Angle Calculation ✅

Manual Conversion ✅

Automatic Conversion ✅

Dataset Structure Creation ✅

Current Project Status

Phase 1 Dataset Understanding ✅

Phase 2 Dataset Preparation ✅

Phase 3 Dataset Visualization ✅

Phase 4 OBB Mathematics ✅

Next Phase

Phase 5 - PyTorch Dataset Class

Objectives:

1. Convert dataset structure into a PyTorch Dataset.
2. Implement len().
3. Implement getitem().
4. Prepare data for DataLoader.
5. Create training-ready batches.
6. Build the data pipeline required for Faster R-CNN training.

Phase 5 marks the transition from data preparation to deep learning model development.

Faster R-CNN OBB Project - Phase 5 Notes

PyTorch Dataset and DataLoader Pipeline

Objective

The objective of this phase is to convert the prepared DOTA dataset into a format that can be used directly by PyTorch for deep learning training.

This phase bridges the gap between:

Dataset Preparation

and

Model Training

At the end of this phase, a complete PyTorch data pipeline is created.

Why Is A Dataset Pipeline Required?

Currently, the dataset exists as:

{

"image_path": "...",

"polygon_boxes": [...],

"rotated_boxes": [...],

"labels": [...]

}

This structure is useful for data storage but cannot be fed directly into a neural network.

PyTorch requires:

Dataset

↓

DataLoader

↓

Mini Batches

↓

Model

Therefore, a custom dataset pipeline must be built.

PyTorch Dataset Architecture

PyTorch datasets follow the structure:

```
class CustomDataset(Dataset):

    def __len__(self):
        pass

    def __getitem__(self, idx):
        pass
```

Every dataset must implement these two functions.

Understanding len()

Purpose:

Returns the total number of samples in the dataset.

Example:

len(dataset)

Output:

458

Meaning:

458 Images Available

Understanding getitem()

Purpose:

Returns a single training sample.

Example:

dataset[0]

Output:

Image

Bounding Boxes

Labels

This function is called repeatedly by PyTorch during training.

Step 1 - Install PyTorch

Goal:

Install PyTorch and Torchvision.

Command

pip install torch torchvision

Installed Versions

torch 2.12.0

torchvision 0.27.0

pillow 12.2.0

Purpose Of PyTorch

PyTorch provides:

Neural Networks

Automatic Differentiation

GPU Training

Tensor Operations

Data Loading Utilities

It is the framework used to implement Faster R-CNN.

Step 2 - Create Dataset Class

File:

dota_dataset.py

Initial Dataset Class

```
import os
import cv2

from torch.utils.data import Dataset

class DOTAOBBDataset(Dataset):

    def __init__(self):

        self.image_folder = "data/images"
```

```
    self.image_files = sorted(
        os.listdir(self.image_folder)
    )

def __len__(self):

    return len(self.image_files)

def __getitem__(self, idx):

    image_file = self.image_files[idx]

    image_path = os.path.join(
        self.image_folder,
        image_file
    )

    image = cv2.imread(image_path)

    return image
```

Purpose

This version loads only images.

Output:

Dataset Size: 458

Image Shape: (1023,1147,3)

Understanding Image Shape

Example:

(1023,1147,3)

Meaning:

Height = 1023

Width = 1147

Channels = 3

The three channels correspond to:

Red

Green

Blue

Step 3 - Load Annotations

Goal:

Load object annotations together with images.

Updated Dataset Output

image, boxes, labels = dataset[0]

Output:

Image

Boxes

Labels

Example

Number Of Objects: 55

First Bounding Box:

[937,913,921,912,923,874,940,875]

First Label:

10

Purpose

Each training sample now contains:

Image

Object Coordinates

Class Labels

which is required for object detection.

Step 4 - Tensor Conversion

Problem:

Python lists cannot be processed by neural networks.

Example:

[

[937,913,...]

]

Neural networks require:

torch.tensor(...)

Conversion

```
image = torch.tensor(
    image,
    dtype=torch.float32
)

boxes = torch.tensor(
    boxes,
    dtype=torch.float32
)

labels = torch.tensor(
    labels,
    dtype=torch.long
)
```

Why float32?

Used for:

Images

Coordinates

Regression Targets

Why long?

Used for:

Class Labels

because classification layers expect integer class IDs.

Output

Image Tensor Shape:

torch.Size([1023,1147,3])

Boxes Tensor Shape:

torch.Size([55,8])

Labels Tensor Shape:

torch.Size([55])

Understanding Tensor Shapes

Image:

torch.Size([1023,1147,3])

Meaning:

Height

Width

Channels

Boxes:

torch.Size([55,8])

Meaning:

55 Objects

8 Coordinates Per Object

Coordinates:

x1,y1,x2,y2,x3,y3,x4,y4

Labels:

torch.Size([55])

Meaning:

55 Class Labels

One label for each object.

What Is A Tensor?

A tensor is the fundamental data structure in PyTorch.

Think of it as:

PyTorch Version Of NumPy Arrays

Benefits:

Fast Computation

GPU Support

Neural Network Compatibility

Step 5 - Create DataLoader

Goal:

Automatically create training batches.

Why Not Train One Image At A Time?

Without DataLoader:

dataset[0]

dataset[1]

dataset[2]

This is inefficient.

Instead:

DataLoader

creates batches.

DataLoader Creation

```
loader = DataLoader(
    dataset,
    batch_size=1,
    shuffle=True
)
```

Understanding Parameters

batch_size:

batch_size=1

Meaning:

One Image Per Batch

Used because DOTA images contain different numbers of objects.

Example:

Image A → 55 Objects

Image B → 11 Objects

Image C → 82 Objects

---

shuffle:

shuffle=True

Meaning:

Randomize Image Order

Improves training.

Testing The DataLoader

```
for images, boxes, labels in loader:

    print(images.shape)

    print(boxes.shape)

    print(labels.shape)

    break
```

Output

Image Batch Shape:

torch.Size([1,1327,1164,3])

Boxes Shape:

torch.Size([1,11,8])

Labels Shape:

torch.Size([1,11])

Understanding Batch Shapes

Images:

[1,1327,1164,3]

Meaning:

Batch Size = 1

Height = 1327

Width = 1164

Channels = 3

Boxes:

[1,11,8]

Meaning:

1 Image

11 Objects

8 Coordinates Per Object

Labels:

[1,11]

Meaning:

1 Image

11 Labels

Complete Data Pipeline

The complete pipeline created in this phase is:

DOTA Dataset

↓

Custom Dataset Class

↓

Tensor Conversion

↓

DataLoader

↓

Mini Batch

↓

Ready For Training

Key Learnings

1. **PyTorch uses Dataset and DataLoader abstractions.**
2. **Every Dataset requires:**

__len__()

__getitem__()

3. **Images, coordinates and labels must be converted into tensors.**
4. **Tensors are required for neural network training.**
5. **DataLoader automatically creates batches.**
6. **DataLoader can shuffle data during training.**
7. **The DOTA dataset can now be loaded directly into PyTorch.**
8. **A complete training-ready data pipeline has been created.**

Phase 5 Status

Install PyTorch ✅

Dataset Class ✅

__len__() ✅

__getitem__() ✅

Image Loading ✅

Annotation Loading ✅

Tensor Conversion ✅

DataLoader Creation ✅

Batch Verification ✅

Current Project Status

Phase 1 Dataset Understanding ✅

Phase 2 Dataset Preparation ✅

Phase 3 Dataset Visualization ✅

Phase 4 OBB Mathematics ✅

Phase 5 PyTorch Data Pipeline ✅

Next Phase

Phase 6 - CNN Backbone

Objectives:

1. **Understand Convolutional Neural Networks.**
2. **Understand Feature Extraction.**
3. **Build a CNN Backbone.**
4. **Generate Feature Maps.**
5. **Prepare inputs for the Region Proposal Network (RPN).**
6. **Begin constructing the Faster R-CNN architecture.**

Phase 6 marks the beginning of the actual deep learning model.

Faster R-CNN OBB Project - Phase 6 Notes (Part A)

CNN Backbone and Feature Map Generation

Objective

The objective of this phase is to build the CNN Backbone of the Faster R-CNN architecture.

The backbone is responsible for converting raw satellite images into meaningful feature maps that can be used by the Region Proposal Network (RPN) and detection head.

This marks the beginning of the actual deep learning model development.

Why Do We Need A CNN Backbone?

Input images contain millions of pixel values.

Example:

1023 × 1147 × 3

A neural network cannot directly understand:

Vehicles

Ships

Planes

Buildings

Roads

from raw pixel values.

Therefore, a Convolutional Neural Network (CNN) is used to automatically extract useful features.

Faster R-CNN Architecture Overview

Input Image

↓

CNN Backbone

↓

Feature Maps

↓

Region Proposal Network (RPN)

↓

ROI Pooling

↓

Classifier + Regressor

↓

Final Detection

The CNN Backbone is the first neural network component in the entire Faster R-CNN pipeline.

What Is A CNN?

CNN stands for:

Convolutional Neural Network

A CNN learns:

Edges

Corners

Textures

Shapes

Objects

from images automatically.

Feature Learning Hierarchy

A CNN learns features in stages.

Early Layers

Detect:

Vertical Edges

Horizontal Edges

Corners

Lines

Middle Layers

Detect:

Vehicle Parts

Road Boundaries

Building Structures

Textures

Deep Layers

Detect:

Vehicles

Ships

Aircraft

Storage Tanks

Harbors

Building The Backbone

File Created:

backbone.py

Backbone Architecture

```
import torch
import torch.nn as nn


class SimpleBackbone(nn.Module):

    def __init__(self):
```

```
        super().__init__()

        self.features = nn.Sequential(

            nn.Conv2d(
                3,
                16,
                kernel_size=3,
                padding=1
            ),

            nn.ReLU(),

            nn.MaxPool2d(2),

            nn.Conv2d(
                16,
                32,
                kernel_size=3,
                padding=1
            ),

            nn.ReLU(),

            nn.MaxPool2d(2)
        )

    def forward(self, x):
```

```
        return self.features(x)
```

Understanding Conv2D

Layer:

```
nn.Conv2d(
    3,
    16,
    kernel_size=3,
    padding=1
)
```

Meaning:

Input Channels = 3

Output Channels = 16

Kernel Size   = 3 × 3

Purpose:

Detect Edges

Detect Corners

Detect Patterns

Understanding ReLU

Layer:

nn.ReLU()

Purpose:

Adds non-linearity.

Without ReLU:

Network becomes a simple linear function

and cannot learn complex patterns.

Understanding Max Pooling

Layer:

nn.MaxPool2d(2)

Purpose:

Reduce image size while preserving important information.

Example:

1024 × 1024

↓

512 × 512

Dummy Image Testing

Input:

```
dummy_image = torch.randn(
    1,
    3,
    1024,
    1024
)
```

Meaning:

Batch Size = 1

Channels = 3

Height = 1024

Width = 1024

Output

```
Input Shape :
torch.Size([1,3,1024,1024])

Output Shape :
torch.Size([1,32,256,256])
```

Shape Analysis

Input:

1 Image

3 Channels

1024 × 1024

After First MaxPool:

512 × 512

After Second MaxPool:

256 × 256

Output:

32 Feature Maps

256 × 256 Resolution

What Are Feature Maps?

Feature maps are learned representations of the image.

Instead of storing:

RGB Pixel Values

they store:

Object Information

Edge Information

Texture Information

Shape Information

Feature Map Generation

Output tensor:

torch.Size([1,32,256,256])

Meaning:

Batch Size = 1

32 Feature Maps

Height = 256

Width = 256

The CNN generated 32 different views of the same image.

Feature Map Visualization

File Created:

visualize_features.py

Purpose:

Visualize what the CNN learns.

Pipeline:

DOTA Image

↓

CNN Backbone

↓

Feature Maps

↓

Visualization

Visualized Feature Maps

The first 8 feature maps were displayed.

Observations:

Feature Map 0

Highlighted:

Vehicles

Roads

Buildings

Parking Areas

Feature Map 1

Highlighted:

Object Edges

Road Boundaries

Container Boundaries

Feature Map 2

Mostly inactive.

Observation:

Not every filter activates strongly.

This is normal.

Feature Map 6

Highlighted:

Vehicle Regions

Bus Parking Areas

Object Clusters

Important Observation

The backbone used random weights.

Therefore:

Feature Maps Are Not Yet Trained

The network has never seen:

Ships

Planes

Vehicles

during training.

After training, feature maps become significantly more meaningful.

Why Feature Maps Matter

The next component:

Region Proposal Network (RPN)

does not operate on the original image.

Instead, it operates on:

Feature Maps

Pipeline:

Image
↓
CNN Backbone
↓
Feature Maps

↓

RPN

Thus, feature maps are the input to the object detection stage.

Backbone Workflow

Complete workflow:

Input Image

↓

Conv Layer 1

↓

ReLU

↓

Max Pool

↓

Conv Layer 2

↓

ReLU

↓

Max Pool

↓

32 Feature Maps

Key Learnings

1. **CNNs automatically learn visual features.**

2. **Convolution layers detect patterns.**
3. **ReLU introduces non-linearity.**
4. **MaxPooling reduces spatial dimensions.**
5. **CNNs generate feature maps instead of using raw pixels.**
6. **Feature maps contain object-related information.**
7. **The backbone output shape was:**

torch.Size([1,32,256,256])

8. **Feature maps will be used by the Region Proposal Network.**

Phase 6 (Part A) Status

Understand CNN Backbone ✅

Build CNN Backbone ✅

Conv2D Layers ✅

ReLU Activation ✅

MaxPooling ✅

Forward Pass ✅

Feature Map Generation ✅

Feature Map Visualization ✅

Current Project Status

Phase 1 Dataset Understanding ✅

Phase 2 Dataset Preparation ✅

Phase 3 Dataset Visualization ✅

Phase 4 OBB Mathematics ✅

Phase 5 PyTorch Data Pipeline ✅

Phase 6 CNN Backbone (Part A) ✅

Next Phase

Phase 6 (Part B) - Region Proposal Network (RPN)

Objectives:

1. **Understand Anchor Boxes.**
2. **Generate Region Proposals.**
3. **Classify Object vs Background.**
4. **Predict Bounding Box Offsets.**

5. **Build the first object detection component of Faster R-CNN.**
6. **Produce candidate regions that may contain objects.**

The RPN is the core component that transforms a CNN into an object detector. 🚀

Phase 6B Notes

Region Proposal Network (RPN)

Objective

The objective of this phase is to build the Region Proposal Network (RPN), which is responsible for identifying regions in an image that may contain objects.

The RPN is the first component that transforms a CNN into an object detector.

What Is An RPN?

RPN stands for:

Region Proposal Network

Its purpose is:

Find possible object locations

before classification occurs.

Faster R-CNN Pipeline

Image

↓

CNN Backbone

↓

Feature Maps

↓

RPN

↓

Region Proposals

↓

ROI Pooling

↓

Classifier

↓

Final Detection

Why Is An RPN Needed?

Without an RPN:

Search Entire Image

which is computationally expensive.

Instead:

RPN proposes likely object locations

which greatly speeds up detection.

Anchor Boxes

The RPN starts by generating predefined boxes.

These are called:

Anchor Boxes

Anchor Generation

File:

rpn.py

Code generated anchors over the feature map.

Feature map size:

256 × 256

Locations:

256 × 256
=
65,536

One anchor was generated per location.

Output

Total Anchors: 65536

First Anchor

[-16,-16,16,16]

Meaning:

Center = (0,0)

Size = 32 × 32

Last Anchor

[1004,1004,1036,1036]

Located near the bottom-right of the image.

Purpose Of Anchors

Anchors act as candidate object locations.

The RPN evaluates:

Does this anchor contain an object?

Anchor Visualization

File:

visualize_anchors.py

Output:

Green boxes distributed across the image.

Observation:

RPN searches everywhere

instead of assuming object locations.

RPN Tasks

For every anchor, the RPN performs two tasks.

Task 1 - Classification

Predict:

Object

or

Background

Example:

0.98 → Object

0.03 → Background

Task 2 - Regression

Predict box corrections.

Example:

Current Anchor:

[100,100,132,132]

Actual Object:

[95,98,145,140]

RPN learns:

Move Left

Move Up

Increase Width

Increase Height

This process is called:

Bounding Box Regression

Building The RPN Head

File:

rpn_head.py

Architecture:

Feature Maps

↓

3×3 Conv Layer

↓

ReLU

↓

Classification Layer

↓

Regression Layer

Classification Branch

Layer:

nn.Conv2d(
256,
1,
1
)

Purpose:

Object / Background Prediction

Regression Branch

Layer:

nn.Conv2d(
256,
4,
1
)

Purpose:

Predict:

dx

dy

dw

dh

for anchor adjustment.

RPN Output

Input:

[1,32,256,256]

Classification Output:

[1,1,256,256]

Meaning:

1 Score Per Location

Regression Output:

[1,4,256,256]

Meaning:

4 Regression Values Per Location

Connecting Backbone And RPN

File:

faster_rcnn_stage1.py

Pipeline:

Image
↓
Backbone
↓
Feature Maps
↓
RPN

Output

Feature Maps:
[1,32,256,256]

Classification Scores:
[1,1,256,256]

Bounding Box Deltas:
[1,4,256,256]

Interpretation

Backbone learned features.

RPN evaluated every location.

RPN predicted:

Possible Objects

Bounding Box Adjustments

This is the first stage of Faster R-CNN.

Key Learnings

1. **RPN generates candidate object regions.**
2. **Anchor boxes provide initial object hypotheses.**
3. **RPN performs classification and regression simultaneously.**
4. **Classification predicts object presence.**
5. **Regression adjusts anchor coordinates.**
6. **Backbone and RPN together form the proposal stage of Faster R-CNN.**
7. **The first end-to-end detector pipeline has been constructed.**

Phase 6 Status

CNN Backbone ✅

Feature Maps ✅

Anchor Generation ✅

Anchor Visualization ✅

RPN Head ✅

Classification Branch ✅

Regression Branch ✅

Backbone + RPN Pipeline ✅

Current Project Status

Phase 1 Dataset Understanding ✅

Phase 2 Dataset Preparation ✅

Phase 3 Dataset Visualization ✅

Phase 4 OBB Mathematics ✅

Phase 5 PyTorch Data Pipeline ✅

Phase 6A CNN Backbone ✅

Phase 6B Region Proposal Network ✅

Next Phase

Phase 7 - ROI Pooling and Detection Head

This is where Faster R-CNN takes the proposed regions and performs:

Object Classification

Bounding Box Prediction

Final Detection

After Phase 7, you'll have the complete Faster R-CNN pipeline structure built. 🚀

Phase 7 Notes

ROI Pooling and Detection Head

Objective

The objective of this phase is to convert region proposals into fixed-size feature representations and perform final object classification and OBB prediction.

This phase completes the Faster R-CNN architecture.

Why ROI Pooling Is Needed

RPN proposals have different sizes.

Example:

Proposal A = 100×60

Proposal B = 180×120

Proposal C = 50×40

Neural networks require fixed-size inputs.

Therefore ROI Pooling converts all proposals into:

7 × 7

feature maps.

ROI Pooling

File:

roi_pool.py

Input:

[1,32,256,256]

Output:

[1,32,7,7]

ROI Pooling Operation

Large Proposal

↓

AdaptiveMaxPool2D

↓

7 × 7 Feature Map

Why 7×7?

Advantages:

Fixed Size

Efficient

Preserves Important Features

This size is commonly used in Faster R-CNN.

Detection Head

File:

detection_head.py

Purpose:

Perform final:

Classification

Bounding Box Prediction

Detection Head Architecture

ROI Features

↓

Flatten

↓

Fully Connected Layer

↓

ReLU

↓

Classification Head

Regression Head

Flatten Layer

Input:

[1,32,7,7]

Output:

[1,1568]

Calculation:

32 × 7 × 7

= 1568

Fully Connected Layer

```
Linear(
    1568,
    512
)
```

Purpose:

Learn relationships between extracted features.

Classification Head

Layer:

```
Linear(
    512,
    15
)
```

Output:

[1,15]

Purpose:

Predict one of the 15 DOTA classes.

Dataset Classes

baseball-diamond
basketball-court
bridge
ground-track-field
harbor
helicopter
large-vehicle
plane
roundabout
ship
small-vehicle
soccer-ball-field
storage-tank
swimming-pool
tennis-court

Total:

15 Classes

Regression Head

Layer:

Linear(
512,
5
)

Output:

[1,5]

Purpose:

Predict:

cx

cy

w

h

θ

for Oriented Bounding Boxes.

Detection Head Output

Input:

[1,32,7,7]

Output:

Class Scores:
[1,15]

Bounding Box Prediction:
[1,5]

Complete Faster R-CNN Pipeline

The complete pipeline implemented so far:

Input Image

↓

CNN Backbone

↓

Feature Maps

↓

Anchor Generation

↓

RPN

↓

Region Proposals

↓

ROI Pooling

↓

Detection Head

↓

Class Prediction

OBB Prediction

Key Learnings

1. **ROI Pooling converts variable-sized proposals into fixed-size feature maps.**
2. **AdaptiveMaxPool2D was used to simulate ROI Pooling.**
3. **The Detection Head performs final classification.**
4. **The Detection Head performs OBB regression.**
5. **OBB prediction uses:**

cx

cy

w

h

θ

instead of standard bounding box coordinates.

6. The complete Faster R-CNN architecture has been assembled.

Phase 7 Status

ROI Pooling ✅

Fixed-Size Features ✅

Detection Head ✅

Classification Branch ✅

OBB Regression Branch ✅

End-To-End Pipeline ✅

Current Project Status

Phase 1 Dataset Understanding ✅

Phase 2 Dataset Preparation ✅

Phase 3 Dataset Visualization ✅

Phase 4 OBB Mathematics ✅

Phase 5 PyTorch Data Pipeline ✅

Phase 6 CNN Backbone ✅

Phase 6B Region Proposal Network ✅

Phase 7 ROI Pooling & Detection ✅

Important Reality Check

You now have the architecture skeleton of a Faster R-CNN OBB detector.

What is still missing for a real trainable detector:

Train / Validation Split

Loss Functions

Anchor Matching (IoU)

Target Generation

Training Loop

Optimizer

Model Saving

Inference Pipeline

OBB Visualization

Evaluation Metrics

These would be the next phases if the goal is a detector that actually learns from your 458 DOTA images.

So the next logical phase is:

Phase 8 — Training Preparation & Loss Functions

This is where the network starts learning instead of just producing random outputs. 🚀

Phase 8A Notes

Loss Functions

Objective

The objective of this phase is to understand how a Faster R-CNN model measures prediction errors during training.

A neural network learns by minimizing a numerical value called the loss.

Why Are Loss Functions Needed?

Suppose:

Ground Truth:

large-vehicle

Prediction:

ship

The model is incorrect.

A loss function measures:

How wrong the prediction is.

The training process then attempts to reduce this error.

Types Of Loss In Faster R-CNN

The detector performs two tasks:

Classification

Bounding Box Regression

Therefore two loss functions are required.

Classification Loss

Used for:

Object Category Prediction

Example:

plane

ship

vehicle

Loss Function:

nn.CrossEntropyLoss()

Purpose:

Measures the difference between:

Predicted Class

Actual Class

Regression Loss

Used for:

Bounding Box Prediction

Loss Function:

nn.SmoothL1Loss()

Purpose:

Measures the difference between:

Predicted Coordinates

Actual Coordinates

Example

Predicted:

(100,200,50,30,10)

Ground Truth:

(110,190,55,35,12)

SmoothL1 calculates the coordinate error.

Total Loss

Formula:

$$Loss_{Total} = Loss_{Classification} + Loss_{Regression}$$

Output Obtained

Classification Loss: 0.7679

Regression Loss: 5.9000

Total Loss: 6.6679

Key Learnings

1. **Loss functions measure prediction errors.**
2. **CrossEntropyLoss is used for classification.**

3. SmoothL1Loss is used for regression.
4. Total loss combines both errors.
5. Training attempts to minimize total loss.

Phase 8A Status

Cross Entropy Loss ✅

Smooth L1 Loss ✅

Total Loss ✅

Phase 8B Notes

Intersection over Union (IoU)

Objective

The objective of this phase is to measure the overlap between anchor boxes and ground truth boxes.

What Is IoU?

IoU stands for:

Intersection over Union

It measures how much two boxes overlap.

Formula

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union}$$

IoU Range

0.0 → No Overlap

1.0 → Perfect Overlap

Example

Ground Truth:

[100,100,200,200]

Anchor:

[120,120,220,220]

Output:

IoU = 0.471

Meaning:

47.1% overlap

IoU Thresholds

Typical Faster R-CNN:

IoU > 0.7

↓

Positive Anchor

IoU < 0.3

↓

Negative Anchor

0.3 ≤ IoU ≤ 0.7

↓

Ignore Anchor

Why IoU Is Important

IoU determines:

Which anchors should learn

Which anchors should be ignored

during training.

Key Learnings

1. **IoU measures box overlap.**
2. **IoU values range from 0 to 1.**
3. **High IoU indicates a good anchor.**

4. **Low IoU indicates background.**
5. **IoU is the foundation of anchor matching.**

Phase 8B Status

IoU Formula ✅

IoU Calculation ✅

IoU Interpretation ✅

Phase 8C Notes

Anchor Matching

Objective

The objective of this phase is to label anchors as Positive, Negative or Ignore using IoU.

Why Anchor Matching?

The RPN predicts:

Object

Background

To train the RPN we need correct labels.

Anchor Matching creates those labels.

Matching Rules

Positive:

IoU ≥ 0.7

Label:

1

Negative:

IoU ≤ 0.3

Label:

0

Ignore:

0.3 < IoU < 0.7

Label:

-1

Results Obtained

Anchor 1:

IoU = 0.826

Status = POSITIVE

Anchor 2:

IoU = 0.471

Status = IGNORE

Anchor 3:

IoU = 0.000

Status = NEGATIVE

Purpose

Anchor matching generates:

Training Labels

for the RPN classification branch.

Key Learnings

1. **Anchor matching uses IoU thresholds.**
2. **Positive anchors learn object detection.**
3. **Negative anchors learn background detection.**
4. **Ignore anchors are excluded from training.**
5. **Anchor matching creates classification targets.**

Phase 8C Status

Positive Anchors ✅

Negative Anchors ✅

Ignore Anchors ✅

Anchor Matching ✅

Phase 8D Notes

Target Generation

Objective

The objective of this phase is to create regression targets for positive anchors.

Why Target Generation?

The RPN regression branch predicts:

dx

dy

dw

dh

The network needs correct target values during training.

Regression Parameters

dx:

Horizontal Shift

---

dy:

Vertical Shift

---

dw:

Width Adjustment

---

dh:

Height Adjustment

Example

Anchor:

[95,95,205,205]

Ground Truth:

[100,100,200,200]

Output

dx = 0.0

dy = 0.0

dw = -0.0953

dh = -0.0953

Interpretation

No horizontal movement

No vertical movement

Reduce width slightly

Reduce height slightly

Purpose

Creates:

Regression Targets

for RPN training.

Key Learnings

1. **Positive anchors require regression targets.**
2. **Targets are represented as dx, dy, dw and dh.**
3. **Targets describe how an anchor should move.**
4. **Regression learns transformations rather than coordinates.**

Phase 8D Status

Regression Targets ✅

dx,dy,dw,dh ✅

Target Generation ✅

Phase 8E Notes

Training Loop

Objective

The objective of this phase is to understand how neural networks learn.

Four Steps Of Training

Forward Pass

Loss Calculation

Backward Pass

Weight Update

Forward Pass

outputs = model(inputs)

Purpose:

Generate Predictions

Loss Calculation

loss = criterion(outputs, targets)

Purpose:

Measure Error

Backward Pass

loss.backward()

Purpose:

Compute Gradients

Weight Update

optimizer.step()

Purpose:

Adjust Model Weights

Results Obtained

Epoch 1 Loss: 1.5110

Epoch 2 Loss: 1.5033

Epoch 3 Loss: 1.4957

Epoch 4 Loss: 1.4881

Epoch 5 Loss: 1.4805

Interpretation

Loss decreased continuously.

Meaning:

Model Learning Successfully

Key Learnings

1. **Training occurs through repeated optimization.**
2. **Forward pass generates predictions.**
3. **Loss measures prediction error.**
4. **Backpropagation computes gradients.**

5. **Optimizer updates weights.**
6. **Decreasing loss indicates learning.**

Phase 8E Status

Forward Pass ✅

Loss Calculation ✅

Backward Pass ✅

Weight Update ✅

Training Loop ✅

Phase 8F Notes

Optimizer and Model Saving

Objective

The objective of this phase is to understand optimizers, learning rates and model checkpointing.

Optimizer

Used to update neural network weights.

Example:

optim.Adam(...)

Adam Optimizer

Advantages:

Fast

Stable

Widely Used

Learning Rate

Example:

lr = 0.001

Controls:

Size Of Weight Updates

Model Saving

Function:

torch.save(...)

Purpose:

Save Learned Weights

Model Loading

Function:

load_state_dict(...)

Purpose:

Restore Saved Weights

Results Obtained

Model Saved!

Model Loaded!

Checkpoints

Used during long training sessions.

Example:

checkpoint_epoch_1.pth

checkpoint_epoch_2.pth

checkpoint_epoch_3.pth

Key Learnings

1. **Adam updates network weights.**
2. **Learning rate controls update magnitude.**

3. **Models can be saved to disk.**
4. **Saved models can be reloaded later.**
5. **Checkpoints prevent training progress loss.**

Phase 8F Status

Adam Optimizer ✅

Learning Rate ✅

Model Saving ✅

Model Loading ✅

Checkpoints ✅

Overall Phase 8 Status

Phase 8A Loss Functions ✅

Phase 8B IoU ✅

Phase 8C Anchor Matching ✅

Phase 8D Target Generation ✅

Phase 8E Training Loop ✅

Phase 8F Optimizer & Saving ✅

Now your notes are complete through Phase 8, and we can move cleanly into Phase 9: First Real Detector Training on the DOTA Dataset. 🚀

Phase 9 Notes

First Real Training on DOTA Dataset

Objective

The objective of this phase is to verify that the DOTA dataset can be used for real neural network training.

Until Phase 8, all experiments were performed on:

Dummy Data

Random Inputs

Small Examples

In Phase 9, the model is trained using actual DOTA images and labels.

Why Phase 9 Is Important

Before building a complete Faster R-CNN detector, it is necessary to verify:

Dataset Loading

Label Loading

CNN Forward Pass

Loss Calculation

Backpropagation

Model Saving

are working correctly.

Phase 9A

Classification Dataset

Objective

Create a simplified dataset for initial training.

Instead of:

image
boxes
labels

the dataset returns:

image
single_class_label

Dataset Structure

Input:

Image File

Corresponding Label File

Example:

P0003.png

P0003.txt

Image Processing

Each image is:

Loaded

Resized

Converted To Tensor

Normalized

Resize

Original:

1023 × 1147

Converted To:

224 × 224

Tensor Conversion

Before:

H × W × C

After:

C × H × W

Result:

torch.Size([3,224,224])

Label Extraction

The first object inside the label file is selected.

Example:

small-vehicle

Converted using:

CLASS_TO_ID

Result:

10

Dataset Verification

Output:

Dataset Size: 456

Image Shape:
torch.Size([3,224,224])

Label:
10

Empty Label Files

During dataset inspection:

P2152.txt

P2330.txt

were found to contain:

Only Metadata

No Objects

These files were removed automatically.

Dataset Statistics

Original Images:

458

Usable Images:

456

Skipped Images:

2

Phase 9B

Simple CNN Classifier

Objective

Build a CNN capable of classifying DOTA images into one of the 15 classes.

Architecture

Input:

224 × 224 RGB Image

Feature Extraction

Layer 1:

```
Conv2D(
    3,
    16,
    kernel_size=3
)
```

Activation:

ReLU()

Pooling:

MaxPool2D(2)

Layer 2:

Conv2D(
16,
32,
kernel_size=3
)

Activation:

ReLU()

Pooling:

MaxPool2D(2)

Classification Head

Flatten:

32 × 56 × 56

↓

100352 Features

Fully Connected:

100352 → 128

Output Layer:

128 → 15

Model Verification

Input:

torch.Size([1,3,224,224])

Output:

torch.Size([1,15])

Interpretation

Output contains:

15 Class Scores

corresponding to:

baseball-diamond

basketball-court

bridge

ground-track-field

harbor

helicopter

large-vehicle

plane

roundabout

ship

small-vehicle

soccer-ball-field

storage-tank

swimming-pool

tennis-court

Phase 9C

Real CNN Training

Objective

Train the CNN using actual DOTA images.

Training Components

Dataset:

ClassificationDataset

Model:

SimpleClassifier

Loss Function:

CrossEntropyLoss

Optimizer:

Adam

Learning Rate:

0.001

Training Pipeline

Image

↓

CNN

↓

Class Scores

↓

Cross Entropy Loss

↓

Backpropagation

↓

Weight Update

Training Results

Epoch 1:

Average Loss = 2.3653

Epoch 2:

Average Loss = 2.0272

Epoch 3:

Average Loss = 1.5560

Loss Trend

2.3653

↓

2.0272

↓

1.5560

Interpretation

Loss decreases consistently.

Meaning:

The CNN is successfully learning patterns from DOTA images.

Model Saving

After training:

```
torch.save(
    model.state_dict(),
    "classifier_model.pth"
)
```

Output:

Model Saved!

Generated File

classifier_model.pth

Contains:

Learned Weights

Model Parameters

Bias Values

Key Learnings

1. **Real DOTA images can be loaded successfully.**
2. **Labels are correctly extracted.**
3. **Empty label files can be handled safely.**
4. **CNN architecture works correctly.**
5. **Loss decreases during training.**
6. **Backpropagation updates weights correctly.**
7. **Model checkpoints can be saved.**

Phase 9 Status

Classification Dataset ✅

Dataset Cleaning ✅

Image Preprocessing ✅

CNN Classifier ✅

Forward Pass ✅

Loss Calculation ✅

Backpropagation ✅

Optimizer Update ✅

Real DOTA Training ✅

Loss Decreasing ✅

Model Saving ✅

Overall Project Progress

Phase 1 Dataset Understanding ✅
Phase 2 Dataset Preparation ✅
Phase 3 Visualization ✅
Phase 4 OBB Mathematics ✅
Phase 5 PyTorch Pipeline ✅
Phase 6 CNN Backbone ✅
Phase 6B RPN ✅
Phase 7 ROI + Detection Head ✅
Phase 8 Training Fundamentals ✅
Phase 9 Real DOTA Classification ✅

Next Phase

Phase 10A
Detection Dataset Construction

where we move from:

Image → Single Class

to:

Image → Multiple Boxes + Multiple Classes

which is the first true step toward a complete Faster R-CNN object detector. 🚀

Phase 10 Notes

Detection Dataset Pipeline

Objective

The objective of this phase is to prepare the DOTA dataset for object detection training.

Unlike image classification:

Image
↓
Single Label

Object detection requires:

Image
↓
Multiple Objects

↓

Bounding Boxes

↓

Class Labels

Why Phase 10 Is Important

A Faster R-CNN detector does not learn from a single class label.

It requires:

Image

Bounding Boxes

Object Classes

for every object present in the image.

Phase 10A

Detection Dataset Construction

Objective

Create a dataset that returns:

image
boxes
labels

instead of:

image
single_label

Dataset Format

Each sample returns:

```
(
    image,
    boxes,
    labels
)
```

Image Format

Output:

torch.Size([3,224,224])

Meaning:

3 Channels

224 Height

224 Width

Bounding Boxes

Output:

torch.Size([55,8])

Meaning:

55 Objects

8 Coordinates Per Object

DOTA OBB Format

Each object is represented as:

x1 y1
x2 y2
x3 y3
x4 y4

Example:

937 913
921 912
923 874
940 875

Stored as:

[
937,913,
921,912,
923,874,
940,875
]

Labels

Output:

torch.Size([55])

Meaning:

55 Object Classes

One label per object.

Output Obtained

Image Shape:
torch.Size([3,224,224])

Boxes Shape:
torch.Size([55,8])

Labels Shape:
torch.Size([55])

First Label:
10

Key Learnings

1. **Detection requires multiple boxes per image.**
2. **DOTA annotations use polygon coordinates.**
3. **Each object has one class label.**
4. **Dataset returns image, boxes and labels.**

Phase 10A Status

Detection Dataset ✅

OBB Loading ✅

Label Loading ✅

Multiple Objects ✅

Phase 10B

Detection DataLoader

Objective

Create a DataLoader capable of handling images with different numbers of objects.

Problem

Image 1:

55 Objects

Image 2:

13 Objects

Image 3:

268 Objects

Normal batching fails because:

[55,8]

[13,8]

[268,8]

cannot be stacked together.

Solution

Custom:

collate_fn()

Function.

Custom Collate Function

```
def collate_fn(batch):

    return tuple(
        zip(*batch)
    )
```

Output Obtained

Batch Size: 4

Image 1:

268 Objects

Image 2:
3 Objects

Image 3:
256 Objects

Image 4:
13 Objects

Purpose

Allows:

Variable Number Of Objects

inside a single batch.

Key Learnings

1. **Object detection batches are irregular.**
2. **Different images contain different object counts.**
3. **Custom collate functions solve batching issues.**
4. **Faster R-CNN uses a similar strategy.**

Phase 10B Status

Custom Collate Function ✅

Detection DataLoader ✅

Variable Object Counts ✅

Phase 10C

Detection Targets

Objective

Convert detection annotations into a target dictionary.

Original Format

image
boxes
labels

New Format

image

target = {

"boxes": boxes,

```
    "labels": labels
}
```

Why Use Targets?

Real detection models expect:

images

targets

instead of separate tensors.

Output Obtained

Target Keys:

boxes

labels

Boxes

torch.Size([55,8])

Labels

torch.Size([55])

Key Learnings

1. **Detection models use target dictionaries.**
2. **Boxes and labels are grouped together.**
3. **Target dictionaries simplify training.**

Phase 10C Status

Target Dictionary ✅

Boxes In Targets ✅

Labels In Targets ✅

Phase 10D

Detection Batch Pipeline

Objective

Create batches containing images and target dictionaries.

Output Structure

images = [

image1,

```
    image2
]

targets = [

    {
        "boxes": ...
        "labels": ...
    },

    {
        "boxes": ...
        "labels": ...
    }
]
```

Output Obtained

Number Of Images:
2

Number Of Targets:
2

Image Shape:
torch.Size([3,224,224])

Boxes Shape:
torch.Size([7,8])

Labels Shape:
torch.Size([7])

Purpose

Matches the format expected by Faster R-CNN training.

Key Learnings

1. **Detection batches contain image lists.**
2. **Each image has its own target dictionary.**
3. **Object counts vary across images.**
4. **Batch structure now resembles production object detectors.**

Phase 10D Status

Image Batching ✅

Target Batching ✅

Detection Pipeline ✅

Phase 10 Overall Summary

Pipeline Created

DOTA Dataset

↓

DetectionDataset

↓

DetectionDataLoader

↓

Target Dictionaries

↓

Batched Images

↓

Batched Targets

Key Concepts Learned

OBB Annotations ✅

Multiple Objects Per Image ✅

Detection Dataset ✅

Custom DataLoader ✅

Target Dictionaries ✅

Detection Batching ✅

Phase 10 Status

Phase 10A Detection Dataset ✅

Phase 10B Detection DataLoader ✅

Phase 10C Detection Targets ✅

Phase 10D Detection Batching ✅

Project Progress After Phase 10

Phase 1 Dataset Understanding ✅
Phase 2 Dataset Preparation ✅
Phase 3 Visualization ✅
Phase 4 OBB Mathematics ✅
Phase 5 PyTorch Pipeline ✅
Phase 6 CNN Backbone ✅
Phase 6B RPN ✅
Phase 7 ROI + Detection Head ✅
Phase 8 Training Fundamentals ✅
Phase 9 Real DOTA Classification ✅
Phase 10 Detection Data Pipeline ✅

Next Phase

Phase 11
Real Faster R-CNN Training

where your DOTA dataset will be connected directly to a real Faster R-CNN model and the detector will begin learning object locations and classes. 🚀🔥

Phase 11 Notes

Real Faster R-CNN Integration

Objective

The objective of Phase 11 is to connect the DOTA dataset with a real Faster R-CNN detector and verify that the complete detection pipeline works correctly.

Before this phase:

Dataset ✅

Backbone ✅

RPN ✅

ROI Pooling ✅

Detection Head ✅

All components existed separately.

In Phase 11 they are connected into a single detector.

Why Phase 11 Is Important

This phase transforms the project from:

Learning Individual Components

into:

Building A Real Object Detector

Phase 11A

Faster R-CNN Dataset

Objective

Convert DOTA polygon annotations into the format expected by Faster R-CNN.

Original DOTA Format

Each object:

x1 y1
x2 y2
x3 y3
x4 y4

Example:

937 913
921 912
923 874
940 875

Faster R-CNN Format

Each object:

xmin
ymin
xmax
ymax

Example:

921
874
940
913

Conversion Method

Extract:

xs = coords[0::2]
ys = coords[1::2]

Compute:

xmin = min(xs)
ymin = min(ys)

xmax = max(xs)
ymax = max(ys)

Output Obtained

Image Shape:
torch.Size([3,1023,1147])

Boxes Shape:
torch.Size([55,4])

Labels Shape:
torch.Size([55])

Key Learnings

1. **Faster R-CNN expects horizontal boxes.**
2. **DOTA annotations must be converted.**
3. **Each image contains multiple objects.**
4. **Target format follows PyTorch detection standards.**

Phase 11A Status

FRCNN Dataset ✅

Polygon Conversion ✅

Horizontal Boxes ✅

Phase 11B

Real Faster R-CNN Model

Objective

Create a production-grade Faster R-CNN detector using torchvision.

Model Used

fasterrcnn_resnet50_fpn()

Architecture

Input Image

↓

ResNet50 Backbone

↓

Feature Pyramid Network

↓

Region Proposal Network

↓

ROI Align

↓

Detection Head

↓

Predictions

Number Of Classes

DOTA Classes:

15

Background Class:

1

Total:

16

Predictor Replacement

Original predictor replaced with:

```
FastRCNNPredictor(
    in_features,
    16
)
```

Model Verification

Output:

FastRCNNPredictor(
cls_score:
out_features=16
)

Key Learnings

1. **Real Faster R-CNN can be customized.**
2. **Detection heads can be replaced.**
3. **Class count must include background.**

Phase 11B Status

Faster R-CNN Created ✅

16 Classes Configured ✅

Model Built Successfully ✅

Phase 11C

First Forward Pass

Objective

Verify that images and targets can pass through the detector.

Input

images

targets

Target Structure

```
target = {
    "boxes": boxes,
    "labels": labels
}
```

Forward Pass

```
loss_dict = model(
    images,
    targets
)
```

Output Obtained

loss_classifier : 2.6604

loss_box_reg : 0.1331

loss_objectness : 0.6928

loss_rpn_box_reg : 0.2886

Loss Definitions

Classification Loss

Predicted Class

vs

Actual Class

Output:

2.6604

Box Regression Loss

Predicted Box

vs

Ground Truth Box

Output:

0.1331

Objectness Loss

Produced by:

Region Proposal Network

Measures:

Object

vs

Background

Output:

0.6928

RPN Regression Loss

Measures:

Anchor Adjustment Accuracy

Output:

0.2886

Significance

This was the first successful execution of:

Image

↓

Faster R-CNN

↓

RPN

↓

ROI Align

↓

Detection Head

↓

Detection Losses

Key Learnings

1. **Dataset is compatible with Faster R-CNN.**
2. **Boxes are correctly loaded.**
3. **Labels are correctly loaded.**
4. **Detection losses can be computed.**

Phase 11C Status

Forward Pass Working ✅

Loss Dictionary Generated ✅

Detection Pipeline Working ✅

Phase 11D

Detection Training Pipeline

Objective

Create a training loop for Faster R-CNN.

Components

Dataset:

FRCNNDataset

Model:

FasterRCNN ResNet50 FPN

Optimizer:

Adam

Learning Rate:

0.0001

Training Flow

Image

↓

Forward Pass

↓

Loss Calculation

↓

Backpropagation

↓

Weight Update

↓

Repeat

Training Script

Created:

train_frcnn.py

Current Status

Training loop:

Implemented

CPU execution:

Very Slow

because:

1023 × 1147 Images

ResNet50

FPN

Faster R-CNN

require significant computation.

Planned Execution

GPU Machine

for full-scale training.

Key Learnings

1. **Detection training requires much more computation than classification.**
2. **GPU acceleration is highly recommended.**
3. **Training code is ready for deployment.**

Phase 11D Status

Training Script Created ✅

Optimizer Configured ✅

Backpropagation Added ✅

GPU Training Pending ⏳

Phase 11 Overall Summary

Complete Detection Pipeline

DOTA Dataset

↓

FRCNNDataset

↓

Faster R-CNN

↓

RPN

↓

ROI Align

↓

Detection Head

↓

Detection Losses

Key Concepts Learned

Horizontal Box Conversion ✅

Real Faster R-CNN ✅

ResNet50 Backbone ✅

Feature Pyramid Network ✅

Detection Losses ✅

Forward Pass ✅

Training Pipeline ✅

Phase 11 Status

Phase 11A Dataset Conversion ✅

Phase 11B Faster R-CNN Model ✅

Phase 11C Forward Pass ✅

Phase 11D Training Pipeline ✅
GPU Training Pending ⏳

Project Progress After Phase 11

Phase 1 Dataset Understanding ✅
Phase 2 Dataset Preparation ✅
Phase 3 Visualization ✅
Phase 4 OBB Mathematics ✅
Phase 5 PyTorch Dataset Pipeline ✅
Phase 6 CNN Backbone ✅
Phase 6B RPN ✅
Phase 7 ROI + Detection Head ✅
Phase 8 Training Fundamentals ✅
Phase 9 Real CNN Training ✅
Phase 10 Detection Data Pipeline ✅
Phase 11 Real Faster R-CNN Integration ✅

Next Phase

Phase 12
Inference and Prediction

where the trained Faster R-CNN model will be used to:

Input Image

↓

Detect Objects

↓

Predict Classes

↓

Predict Bounding Boxes

↓

Visualize Results

This is the phase where you'll finally see object detections on images. 🚀